Producer-Consumer Problem

Definition

The **Producer-Consumer Problem** (also called Bounded-Buffer Problem) involves two types of processes sharing a fixed-size buffer. Producers generate data and put it into the buffer, while consumers take data from the buffer.

Key Challenges

- Producer must not add data to a **full buffer**
- Consumer must not remove data from an **empty buffer**
- Access to buffer must be **mutually exclusive**

Visual Representation

text

Producer(s) → [BUFFER] → Consumer(s)

Fixed Size

Put data Remove data

Producer

```
do {
  wait (empty); // wait until empty>0
          and then decrement 'empty'
  wait (mutex); // acquire lock
  /* add data to buffer */
  signal (mutex); // release lock
  signal (full); // increment 'full'
} while(TRUE)
```

Consumer

```
do {
  wait (full); // wait until full>0 and
          then decrement 'full'
  wait (mutex); // acquire lock
  /* remove data from buffer */
  signal (mutex); // release lock
  signal (empty); // increment 'empty'
} while(TRUE)
```

Simple C Code Example

c

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
```

```
#define BUFFER_SIZE 5
#define NUM_ITEMS 10

int buffer[BUFFER_SIZE];
int count = 0;  // Items in buffer
int in = 0;     // Next free position
int out = 0;    // Next full position

sem_t empty;    // Counts empty slots
sem_t full;     // Counts full slots
pthread_mutex_t mutex;  // Protects buffer access

void* producer(void* arg) {
   for(int i = 0; i < NUM_ITEMS; i++) {
      // Wait for empty slot
      sem_wait(&empty);

      // Lock buffer
      pthread_mutex_lock(&mutex);

      // CRITICAL SECTION: Add item
      buffer[in] = i;
      printf("Produced: %d at position %d\n", i, in);
      in = (in + 1) % BUFFER_SIZE;
      count++;

      // Unlock buffer
      pthread_mutex_unlock(&mutex);

      // Signal that buffer has data
      sem_post(&full);
```

```
    }
    return NULL;
}

void* consumer(void* arg) {
    for(int i = 0; i < NUM_ITEMS; i++) {
        // Wait for full slot
        sem_wait(&full);

        // Lock buffer
        pthread_mutex_lock(&mutex);

        // CRITICAL SECTION: Remove item
        int item = buffer[out];
        printf("Consumed: %d from position %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        count--;

        // Unlock buffer
        pthread_mutex_unlock(&mutex);

        // Signal that buffer has empty slot
        sem_post(&empty);
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);  // Initially all slots empty
```

```
    sem_init(&full, 0, 0);          // Initially no slots full
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    return 0;
}
```

Real-World Examples

- **Print spooler**: Multiple apps produce print jobs, printer consumes them
- **Web servers**: Request queue between web servers and application servers
- **Streaming**: Video buffer between producer (streaming server) and consumer (viewer)

---

2. Readers-Writers Problem

Definition

The **Readers-Writers Problem** involves a shared data resource where:

- **Readers** only read the data (don't modify)
- **Writers** modify the data
- Multiple readers can access simultaneously
- Only one writer can access at a time
- No reader should access while a writer is writing

Key Challenges

- **Reader priority**: If readers keep coming, writers may starve
- **Writer priority**: Once a writer is ready, no new readers should start
- **Fairness**: Balance between readers and writers

Visual Representation

text

Readers (R1, R2, R3) → Can read simultaneously

↓

[SHARED DATA]

↓

Writers (W1, W2) → Only one can write at a time

Writer Process

do {
/* writer requests for critical section */
wait(wrt);
/* performs the write */
// leaves the critical section
signal(wrt);
} while(true);

Reader Process

do {
wait (mutex);
readcnt++; // The number of readers has now increased by 1
if (readcnt==1)
wait (wrt); // this ensure no writer can enter if there is even one reader
signal (mutex); // other readers can enter while this current reader is inside the critical section
/* current reader performs reading here */
wait (mutex);
readcnt--; // a reader wants to leave
if (readcnt == 0) //no reader is left in the critical section
signal (wrt); // writers can enter
signal (mutex); // reader leaves
} while(true);

Simple C Code Example (Reader Priority)

c

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t write_lock = PTHREAD_MUTEX_INITIALIZER;
int reader_count = 0;
int shared_data = 1000;  // Shared resource

void* reader(void* arg) {
    int id = *(int*)arg;
```

```
    for(int i = 0; i < 3; i++) {
        // ENTRY SECTION: Update reader count
        pthread_mutex_lock(&mutex);
        reader_count++;
        if(reader_count == 1) {
            pthread_mutex_lock(&write_lock);  // First reader locks writers out
        }
        pthread_mutex_unlock(&mutex);

        // CRITICAL SECTION: Reading
        printf("Reader %d reads: %d (active readers: %d)\n",
               id, shared_data, reader_count);
        sleep(1);  // Simulate reading time

        // EXIT SECTION: Decrease reader count
        pthread_mutex_lock(&mutex);
        reader_count--;
        if(reader_count == 0) {
            pthread_mutex_unlock(&write_lock);  // Last reader allows writers
        }
        pthread_mutex_unlock(&mutex);

        sleep(rand() % 2);
    }
    return NULL;
}

void* writer(void* arg) {
    int id = *(int*)arg;
```

```
    for(int i = 0; i < 2; i++) {
        // ENTRY SECTION: Wait for exclusive access
        pthread_mutex_lock(&write_lock);

        // CRITICAL SECTION: Writing
        shared_data += 100;
        printf("Writer %d writes: %d\n", id, shared_data);
        sleep(2);  // Simulate writing time

        // EXIT SECTION: Release access
        pthread_mutex_unlock(&write_lock);

        sleep(rand() % 3);
    }
    return NULL;
}

int main() {
    pthread_t readers[3], writers[2];
    int reader_ids[3] = {1, 2, 3};
    int writer_ids[2] = {1, 2};

    // Create threads
    for(int i = 0; i < 3; i++)
        pthread_create(&readers[i], NULL, reader, &reader_ids[i]);
    for(int i = 0; i < 2; i++)
        pthread_create(&writers[i], NULL, writer, &writer_ids[i]);

    // Wait for completion
    for(int i = 0; i < 3; i++)
        pthread_join(readers[i], NULL);
```

```
    for(int i = 0; i < 2; i++)
        pthread_join(writers[i], NULL);

    return 0;
}
```

Variations

1. First Readers-Writers Problem (Reader Priority)

- Readers don't wait unless a writer has already obtained write lock
- Can cause writer starvation

2. Second Readers-Writers Problem (Writer Priority)

- Once a writer is ready, no new readers can start
- Prevents writer starvation but can cause reader starvation

3. Third Readers-Writers Problem (Fairness)

- Uses a queue to ensure alternating access
- No starvation for either readers or writers

Real-World Examples

- **Database systems**: Multiple users reading, few updating
- **File systems**: Multiple processes reading, few writing
- **Configuration management**: Many readers, occasional updates
- **Cache systems**: Multiple reads, periodic cache updates

What is the Dining Philosophers Problem?

The Dining Philosophers problem is a classic synchronization problem conceived by Edsger Dijkstra in 1965. It illustrates the challenges of resource allocation, deadlock, and starvation in concurrent systems.

Scenario:

- Five philosophers sit around a circular table
- Each philosopher has a plate of spaghetti
- There is one fork between each pair of philosophers (5 forks total)
- Each philosopher alternates between thinking and eating
- To eat, a philosopher needs both forks (the left one and the right one)
- After eating, they put down both forks and return to thinking

Constraints:

- Forks are shared resources that must be mutually exclusive
- No two adjacent philosophers can eat simultaneously
- The solution must avoid:
    - Deadlock: All philosophers hold one fork and wait forever
    - Starvation: A philosopher never gets to eat

Visual Representation

Example Scenario

Problematic Scenario (Deadlock):

1. All philosophers become hungry at the same time
2. Each picks up their left fork simultaneously
3. Now each waits for their right fork, which is held by the neighbor
4. System deadlocks — no one can eat forever

C Code --- Using Mutex Locks – Having deadlocks

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_PHILOSOPHERS 5

pthread_mutex_t forks[NUM_PHILOSOPHERS];
pthread_t philosophers[NUM_PHILOSOPHERS];

void* philosopher(void* num) {
   int id = *(int*)num;
   int left_fork = id;
   int right_fork = (id + 1) % NUM_PHILOSOPHERS;

   while (1) {
      // Thinking
      printf("Philosopher %d is thinking\n", id);
      usleep(1000000);

      // Pick up forks
      pthread_mutex_lock(&forks[left_fork]);
      printf("Philosopher %d picked up left fork %d\n", id, left_fork);
```

```
        pthread_mutex_lock(&forks[right_fork]);
        printf("Philosopher %d picked up right fork %d\n", id, right_fork);

        // Eating
        printf("Philosopher %d is eating\n", id);
        usleep(1000000);

        // Put down forks
        pthread_mutex_unlock(&forks[right_fork]);
        pthread_mutex_unlock(&forks[left_fork]);
        printf("Philosopher %d finished eating\n", id);
    }
    return NULL;
}

int main() {
    int ids[NUM_PHILOSOPHERS];

    // Initialize mutexes
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_init(&forks[i], NULL);
    }

    // Create philosopher threads
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    // Wait for threads (infinite loop)
```

```
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(philosophers[i], NULL);
    }

    // Cleanup
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}
```

Solution 2: Deadlock Prevention (Different Fork Pickup Order)

```
c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_PHILOSOPHERS 5

pthread_mutex_t forks[NUM_PHILOSOPHERS];
pthread_t philosophers[NUM_PHILOSOPHERS];

void* philosopher(void* num) {
    int id = *(int*)num;
    int first_fork, second_fork;

    // Prevent deadlock: last philosopher picks up right fork first
    if (id == NUM_PHILOSOPHERS - 1) {
        first_fork = (id + 1) % NUM_PHILOSOPHERS;  // Right fork
```

```
        second_fork = id;                    // Left fork
    } else {
        first_fork = id;                     // Left fork
        second_fork = (id + 1) % NUM_PHILOSOPHERS; // Right fork
    }

    while (1) {
        // Thinking
        printf("Philosopher %d is thinking\n", id);
        usleep(1000000);

        // Pick up forks in specified order
        pthread_mutex_lock(&forks[first_fork]);
        printf("Philosopher %d picked up fork %d\n", id, first_fork);

        pthread_mutex_lock(&forks[second_fork]);
        printf("Philosopher %d picked up fork %d\n", id, second_fork);

        // Eating
        printf("Philosopher %d is eating\n", id);
        usleep(1000000);

        // Put down forks
        pthread_mutex_unlock(&forks[second_fork]);
        pthread_mutex_unlock(&forks[first_fork]);
        printf("Philosopher %d finished eating\n", id);
    }
    return NULL;
}

int main() {
```

```
    int ids[NUM_PHILOSOPHERS];

    // Initialize mutexes
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_init(&forks[i], NULL);
    }
    // Create philosopher threads
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    // Wait for threads
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(philosophers[i], NULL);
    }
    // Cleanup
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}
```

Monitor Solution to Dining Philosophers Problem

A monitor-based solution provides a clean, structured approach to the Dining Philosophers problem by encapsulating resource management and using condition variables to control access.

Concept Overview

A monitor is a high-level synchronization construct that provides:

- **Mutual exclusion:** Only one process can execute inside the monitor at a time
- **Condition variables:** Allow processes to wait for specific conditions and signal others
- **State tracking:** Maintains philosopher states (thinking, hungry, eating)

Key Idea: A philosopher can only eat when **both neighbors are not eating**. If forks are unavailable, they wait on a condition variable.

A monitor solution to the dining-philosopher problem

```
monitor dp {
        enum { THINKING, HUNGRY, EATING } state [5];
        condition self [5];

        void pickup (int i) {
                state [i] = HUNGRY;
                test (i);
                if (state [i] != EATING)
                self [i].wait();
                }
        void putdown(int i) {
                state [i] = THINKING;
                test ((i + 4) % 5);
                test ((i + 1) % 5);
                }

        void test (int i) {
                if ((state [(i + 4) % 5] != EATING) &&
                        (state [i] == HUNGRY) &&
                        (state [i + 1] % 5] != EATING)) {
                                state [i] = EATING;
                                self [i].signal();
                                }
                        }
        initialization-code () {
                for (int i = 0; i< 5; i++)
                state [i] = THINKING;
                }
}
```

Philosopher States

c

```
typedef enum {
   THINKING,
   HUNGRY,
   EATING
} PhilosopherState;
```

- **THINKING:** Philosopher is not interested in eating
- **HUNGRY:** Philosopher wants to eat but is waiting for forks
- **EATING:** Philosopher has both forks and is eating

Monitor Structure

c

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_PHILOSOPHERS 5

// Monitor structure
typedef struct {
    PhilosopherState state[NUM_PHILOSOPHERS];
    pthread_mutex_t mutex;              // Monitor mutex
    pthread_cond_t condition[NUM_PHILOSOPHERS]; // One CV per philosopher
} DiningMonitor;

DiningMonitor monitor;
pthread_t philosophers[NUM_PHILOSOPHERS];
```

Complete C Implementation

c

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h>

#define NUM_PHILOSOPHERS 5
#define THINK_TIME 1000000
#define EAT_TIME 1000000

typedef enum {
```

```
    THINKING,
    HUNGRY,
    EATING
} PhilosopherState;

typedef struct {
    PhilosopherState state[NUM_PHILOSOPHERS];
    pthread_mutex_t mutex;              // Monitor mutex (for mutual exclusion)
    pthread_cond_t condition[NUM_PHILOSOPHERS]; // Condition variables
} DiningMonitor;

DiningMonitor monitor;

// Function prototypes
void monitor_init();
void monitor_take_forks(int philosopher_id);
void monitor_put_forks(int philosopher_id);
void test(int philosopher_id);

void* philosopher(void* num);
int left(int philosopher_id);
int right(int philosopher_id);

// Initialize monitor
void monitor_init() {
    pthread_mutex_init(&monitor.mutex, NULL);

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        monitor.state[i] = THINKING;
        pthread_cond_init(&monitor.condition[i], NULL);
    }
```

```
}

// Check if philosopher can start eating
void test(int philosopher_id) {
    // Condition: philosopher is hungry AND neither neighbor is eating
    if (monitor.state[philosopher_id] == HUNGRY &&
        monitor.state[left(philosopher_id)] != EATING &&
        monitor.state[right(philosopher_id)] != EATING) {

        monitor.state[philosopher_id] = EATING;

        // Signal that forks are now available
        pthread_cond_signal(&monitor.condition[philosopher_id]);
    }
}

// Philosopher attempts to pick up forks
void monitor_take_forks(int philosopher_id) {
    pthread_mutex_lock(&monitor.mutex);

    // Mark philosopher as hungry
    monitor.state[philosopher_id] = HUNGRY;

    // Try to acquire forks
    test(philosopher_id);

    // If not eating, wait until condition is signaled
    while (monitor.state[philosopher_id] != EATING) {
        pthread_cond_wait(&monitor.condition[philosopher_id],
                &monitor.mutex);
    }
```

```
    pthread_mutex_unlock(&monitor.mutex);
}

// Philosopher puts down forks
void monitor_put_forks(int philosopher_id) {
    pthread_mutex_lock(&monitor.mutex);

    // Mark philosopher as thinking
    monitor.state[philosopher_id] = THINKING;

    // Test neighbors to see if they can now eat
    test(left(philosopher_id));
    test(right(philosopher_id));

    pthread_mutex_unlock(&monitor.mutex);
}

// Helper functions to get neighbor indices
int left(int philosopher_id) {
    return (philosopher_id + NUM_PHILOSOPHERS - 1) % NUM_PHILOSOPHERS;
}

int right(int philosopher_id) {
    return (philosopher_id + 1) % NUM_PHILOSOPHERS;
}

// Philosopher thread function
void* philosopher(void* num) {
    int id = *(int*)num;
```

```
    while (1) {
        // Thinking
        printf("Philosopher %d is THINKING\n", id);
        usleep(THINK_TIME);

        // Get hungry and pick up forks
        printf("Philosopher %d is HUNGRY\n", id);
        monitor_take_forks(id);

        // Eating
        printf("Philosopher %d is EATING\n", id);
        usleep(EAT_TIME);

        // Put down forks
        printf("Philosopher %d finished EATING\n", id);
        monitor_put_forks(id);
    }

    return NULL;
}

int main() {
    int ids[NUM_PHILOSOPHERS];

    // Initialize monitor
    monitor_init();

    // Create philosopher threads
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
```

```
    }

    // Wait for threads (infinite)
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(philosophers[i], NULL);
    }

    return 0;
}
```